

得到下凸线之后，对于任何一个点 P_t 来说，最优点 P_t 都在切点，如图 8-18 所示。

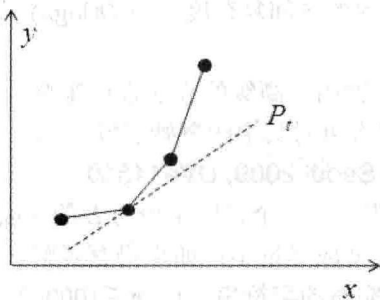


图 8-18 最优点 P_t

如何求切点呢？随着 t 的增大，斜率也是越来越大，所以每次求出的 t' 只会增大，不会减小。因此每次增加到斜率变小时停下来即可。时间复杂度为 $O(n)$ 。

8.6 竞赛题目选讲

例题 8-10 抄书 (Copying Books, UVa 714)

把一个包含 m 个正整数的序列划分成 k 个 ($1 \leq k \leq m \leq 500$) 非空的连续子序列，使得每个正整数恰好属于一个序列。设第 i 个序列的各数之和为 $S(i)$ ，你的任务是让所有 $S(i)$ 的最大值尽量小。例如，序列 1 2 3 2 5 4 划分成 3 个序列的最优方案为 1 2 3 | 2 5 | 4，其中 $S(1)$ 、 $S(2)$ 、 $S(3)$ 分别为 6、7、4，最大值为 7；如果划分成 1 2 | 3 2 | 5 4，则最大值为 9，不如刚才的好。每个整数不超过 10^7 。如果有多解， $S(1)$ 应尽量小。如果仍然有多解， $S(2)$ 应尽量小，依此类推。

【分析】

“最大值尽量小”是一种很常见的优化目标。下面考虑一个新的问题：能否把输入序列划分成 m 个连续的子序列，使得所有 $S(i)$ 均不超过 x ？将这个问题的答案用谓词 $P(x)$ 表示，则让 $P(x)$ 为真的最小 x 就是原题的答案。 $P(x)$ 并不难计算，每次尽量往右划分即可（想一想，为什么）。

接下来又可以猜数字了——随便猜一个 x_0 ，如果 $P(x_0)$ 为假，那么答案比 x_0 大；如果 $P(x_0)$ 为真，则答案小于或等于 x_0 。至此，解法已经得出：二分最小值 x ，把优化问题转化为判定问题 $P(x)$ 。设所有数之和为 M ，则二分次数为 $O(\log M)$ ，计算 $P(x)$ 的时间复杂度为 $O(n)$ （从左到右扫描一次即可），因此总时间复杂度为 $O(n \log M)$ 。

例题 8-11 全部相加 (Add All, UVa 10954)

有 n ($n \leq 5000$) 个数的集合 S ，每次可以从 S 中删除两个数，然后把它们的和放回集合，直到剩下一个数。每次操作的开销等于删除的两个数之和，求最小总开销。所有数均小于 10^5 。

【分析】

这不就是 Huffman 编码的建立过程吗？因为 n 比较小，还可以采用一种更容易写的方

法——使用一个优先队列。

```
#include<cstdio>
#include<queue>
using namespace std;

int main() {
    int n, x;
    while(scanf("%d", &n) == 1 && n) {
        priority_queue<int, vector<int>, greater<int> > q;
        for(int i = 0; i < n; i++) { scanf("%d", &x); q.push(x); }
        int ans = 0;
        for(int i = 0; i < n-1; i++) {
            int a = q.top(); q.pop();
            int b = q.top(); q.pop();
            ans += a+b;
            q.push(a+b);
        }
        printf("%d\n", ans);
    }
    return 0;
}
```

例题 8-12 奇怪的气球膨胀 (Erratic Expansion, UVa12627)

一开始有一个红气球。每小时后，一个红气球会变成 3 个红气球和一个蓝气球，而一个蓝气球会变成 4 个蓝气球，如图 8-19 所示分别是经过 0, 1, 2, 3 小时后的情况。经过 k 小时后，第 $A \sim B$ 行一共有多少个红气球？例如， $k=3$ ， $A=3$ ， $B=7$ ，答案为 14。

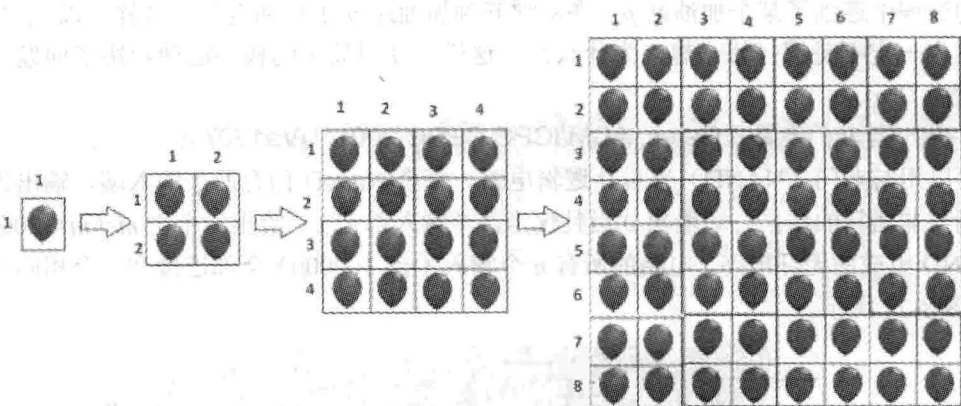


图 8-19 奇怪的气球膨胀示意图

【分析】

如图 8-20 所示， k 小时的情况由 4 个 $k-1$ 小时的情况拼成，其中右下角全是蓝气球，不用考虑。剩下的 3 个部分有一个共同点：都是前 $k-1$ 小时后“最下面若干行”或者“最

上面若干行”的红气球总数。

具体来说，设 $f(k, i)$ 表示 k 小时之后最上面 i 行的红气球总数， $g(k, i)$ 表示 k 小时之后最下面 i 行的红气球总数（规定 $i \leq 0$ 时 $f(k, i) = g(k, i) = 0$ ），则所求答案为 $f(k-1, B-2^{k-1}) + 2 * g(k-1, 2^{k-1} - A + 1)$ 。

如何计算 $f(k, i)$ 和 $g(k, i)$ 呢？以 $g(k, i)$ 为例，下面分两种情况进行讨论，如图 8-21 所示。

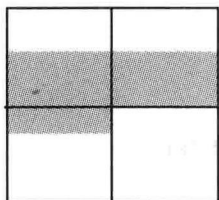


图 8-20 k 小时的情况

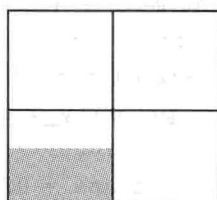
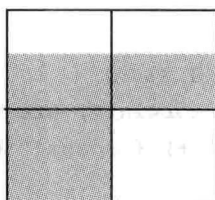


图 8-21 计算 $g(k, i)$

如果 $i \geq 2^{k-1}$ ，则 $g(k, i) = 2g(k-1, i-2^{k-1}) + c(k)$ ，否则 $g(k, i) = g(k-1, i)$ 。其中， $c(k)$ 表示 k 小时后红气球的总数，满足递推式 $c(k) = 3c(k-1)$ ，而 $c(0) = 1$ ，因此 $c(k) = 3^k$ 。

不管是哪种情况， $g(k, i)$ 都可以直接转化为 $k-1$ 的情况，因此 $g(k, i)$ 的计算时间为 $O(k)$ 。类似地， $f(k, i)$ 的计算时间也是 $O(k)$ ，因此本题的总时间复杂度为 $O(k)$ 。

例题 8-13 环形跑道（Just Finish it up, UVa 11093）

环形跑道上有 n ($n \leq 100000$) 个加油站，编号为 $1 \sim n$ 。第 i 个加油站可以加油 p_i 加仑。从加油站 i 开到下一站需要 q_i 加仑汽油。你可以选择一个加油站作为起点，初始油箱为空（但可以立即加油）。你的任务是选择一个起点，使得可以走完一圈后回到起点。假定油箱中的油量没有上限。如果无解，输出 Not possible，否则输出可以作为起点的最小加油站编号。

【分析】

考虑 1 号加油站，直接模拟判断它是否为解。如果是，直接输出；如果不是，说明在模拟的过程中遇到了某个加油站 p ，在从它开到加油站 $p+1$ 时油没了。这样，以 $2, 3, \dots, p$ 为起点也一定不是解（想一想，为什么）。这样，使用简单的枚举法便解决了问题，时间复杂度为 $O(n)$ 。

例题 8-14 与非门电路（Gates, ACM/ICPC CERC 2001, UVa1607）

可以用与非门（NAND）来设计逻辑电路。每个 NAND 门有两个输入端，输出为两个输入端与非运算的结果。即输出 0 当且仅当两个输入都是 1。给出一个由 m ($m \leq 200000$) 个 NAND 组成的无环电路，电路的所有 n 个输入 ($n \leq 100000$) 全部连接到一个相同的输入 x ，如图 8-22 所示。

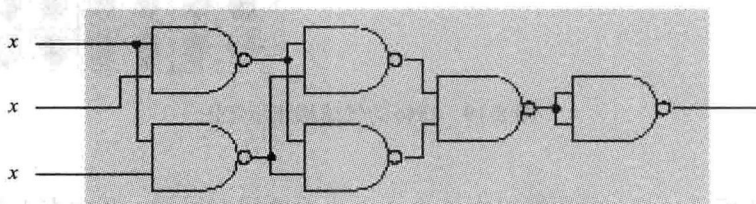


图 8-22 与非门输入电路

请把其中一些输入设置为常数，用最少的 x 完成相同功能。输出任意方案即可。如图 8-23 所示是一个只用一个 x 输入但是可以得到同样结果的电路。

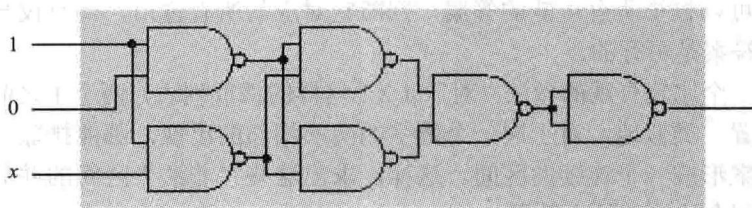


图 8-23 只用一个 x 输入

【分析】

因为只有一个输入 x ，所以整个电路的功能不外乎 4 种：常数 0、常数 1、 x 及非 x 。先把 x 设为 0，再把 x 设为 1，如果二者的输出相同，整个电路肯定是常数，任意输出一种方案即可。

如果 $x=0$ 和 $x=1$ 的输出不同，说明电路的功能是 x 或者非 x ，解至少等于 1。不妨设 $x=0$ 时输出 0， $x=1$ 时输出 1。现在把第一个输入改成 1，其他仍设为 0（记这样的输入为 $1000\cdots 0$ ），如果输出是 1，则得到了一个解 $x000\cdots 0$ 。

如果 $1000\cdots 0$ 的输出也是 0，再把输入改成 $1100\cdots 0$ ，如果输出是 1，则又得到了一个解 $1x00\cdots 0$ 。如果输出还是 0，再尝试 $1110\cdots 0$ ，如此等等。由于输入全 1 时输出为 1，这个算法一定会成功。

问题在于 m 太大，而每次“给定输入计算输出”都需要 $O(m)$ 时间，逐个尝试会很慢。好在已经学习了二分查找：只需二分 1 的个数，即可在 $O(\log m)$ 次计算之内得到结果，总时间复杂度为 $O(m \log m)$ 。

例题 8-15 Shuffle 的播放记录 (Shuffle, ACM/ICPC NWERC 2008, UVa 12174)

你正在使用的音乐播放器有一个所谓的乱序功能，即随机打乱歌曲的播放顺序。假设一共有 s 首歌，则一开始会给这 s 首歌随机排序，全部播放完后再重新随机排序、继续播放，依此类推。注意，当 s 首歌播放完毕之前不会重新排序。这样，播放记录里的每 s 首歌都是 $1\sim s$ 的一个排列。

给出一个长度为 n ($1 \leq s, n \leq 100000$) 的播放记录（不一定是从最开始记录的） x_i ($1 \leq x_i \leq s$)，你的任务是统计下次随机排序所发生的时间有多少种可能性。

例如， $s=4$ ，播放记录是 3, 4, 4, 1, 3, 2, 1, 2, 3, 4，不难发现只有一种可能性：前两首是一个段的最后两首歌，后面是两个完整的段，因此答案是 1；当 $s=3$ 时，播放记录 1, 2, 1 有两种可能：第一首是一个段，后两首是另一段；前两首是一段，最后一首是另一段。答案为 2。

【分析】

“连续的 s 个数”让你联想到了什么？没错，滑动窗口！这次的窗口大小是“基本”固定的（因为还需要考虑不完整的段），因此只需要一个指针；而且所有数都是 $1\sim s$ 的整数，也不需要 STL 的 set，只需要一个数组即可保存每个数在窗口中出现的次数。再用一个

变量记录在窗口中恰好出现一次的数的个数, 则可以在 $O(n)$ 时间内判断出每个窗口是否满足要求 (每个整数最多出现一次)。

这样, 就可以枚举所有可能的答案, 判断它对应的所有窗口, 当且仅当所有窗口均满足要求时这个答案是可行的。

本题还有一个比较直观的做法: 对于 1 2 1 这样的播放列表, 两个 1 之间必然存在一个窗口的交界位置。类似地, 对于同一个数字的两次相邻的出现, 都能排除一些答案, 而且排除的那些答案形成一个连续的区间。这样, 求出这些“非法”区间的并集, 然后求出总长度, 就能得到合法答案的个数了。

例题 8-16 不无聊的序列 (Non-boring sequences, CERC 2012, UVa1608)

如果一个序列的任意连续子序列中至少有一个只出现一次的元素, 则称这个序列是不无聊 (non-boring) 的。输入一个 n ($n \leq 200000$) 个元素的序列 A (各个元素均为 10^9 以内的非负整数), 判断它是不是不无聊的。

【分析】

不难想到整体思路: 在整个序列中找一个只出现一次的元素, 如果不存在, 则这个序列不是不无聊的; 如果找到一个只出现一次的元素 $A[p]$, 则只需检查 $A[1 \cdots p-1]$ ^① 和 $A[p+1 \cdots n]$ 是否满足条件 (想一想, 为什么)。设长度为 n 的序列需要 $T(n)$ 时间, 则有 $T(n) = \max\{T(k-1) + T(n-k) + \text{找到唯一元素 } k \text{ 的时间}\}$ 。这里取 $\max$ 是因为要看最坏情况。

如何找唯一元素? 如果事先算出每个元素左边和右边最近的相同元素 (还记得《唯一的雪花》吗?), 则可以在 $O(1)$ 时间内判断在任意一个连续子序列中, 某个元素是否唯一。如果从左边找, 最坏情况下唯一元素是最后一个元素, 因此

$$T(n) = T(n-1) + O(n) \geq T(n) = O(n^2)$$

从右往左找也一样, 只不过最坏情况变成了“唯一元素是第一个元素”, 但时间复杂度不变。那么, 从两边往中间找会怎样? 此时 $T(n) = \max\{T(k) + T(n-k) + \min(k, n-k)\}$, 刚才的最坏情况 (即第一个元素或最后一个元素是唯一元素) 变成了 $T(n) = T(n-1) + O(1)$ (因为一下子就找到唯一元素了), 即 $T(n) = O(n)$ 。而此时的最坏情况是唯一元素在中间的情况, 它满足经典递推式 $T(n) = 2T(n/2) + O(n)$, 即 $T(n) = O(n \log n)$ 。

例题 8-17 不公平竞赛 (Foul Play, ACM/ICPC NWERC 2012, UVa1609)

n 支队伍 ($2 \leq n \leq 1024$, 且 n 是 2 的整数幂) 打淘汰赛, 每轮都是两两配对, 胜者进入下一轮, 如图 8-24 所示。

每支队伍的实力固定, 并且已知每两支队伍之间的一场比赛结果 (“实力固定”是指, 例如, 队伍 1 曾经胜过队伍 2, 则二者在今后的交锋中队伍 1 总会获胜)。你喜欢 1 号队。虽然它不一定是最强的, 但是它可以直接打败其他队伍中的至少一半, 并且对于每支 1 号队不能直接打败的队伍 t , 总是存在一支 1 号队能直接打败的队伍 t' 使得 t' 能直接打败 t 。问: 是否存在一种比赛安排, 使得 1 号队夺冠?

【分析】

首先从简单情况分析。 $n=2$ 时, 只有 1 号队伍和另外一支队伍。1 号队伍肯定能打败对

^① $A[1 \cdots p-1]$ 表示子序列 $A[1], A[2], \dots, A[p-1]$ 。

手，因为1号队伍能打败至少一半的队伍，此时“一半的队伍”就是这个唯一的对手。

注意到 n 是2的整数幂，所以每次都会恰好淘汰一半的队伍。如果能设计一轮赛程，使得比赛之后所有队伍的情况仍然满足题目的两个条件，则 $\log_2 n$ 次之后1号队伍夺冠。由于这两个条件非常重要，下面给它们编号。

条件1：1号队能直接打败一半的队伍。

条件2：对于不能直接打败的队伍 t ，存在队伍 t' 使得1号队能打败 t' ，且 t' 能打败 t 。

用黑色代表强队（即1号队不能直接打败的队伍），再用灰色代表“有用的队”，即能打败某个黑色队但不能打败1号队的队伍（说它们有用是因为可以间接打败黑色队），最后用问号代表1号队能打败的队伍（可能是灰色也可能不是，但一定不是黑色）。将赛程安排分为4个阶段，如图8-25所示。

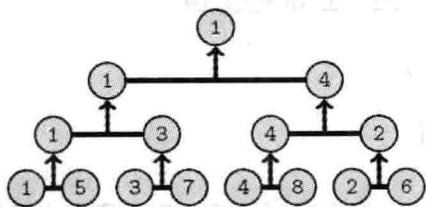


图 8-24 不公平竞赛示意图

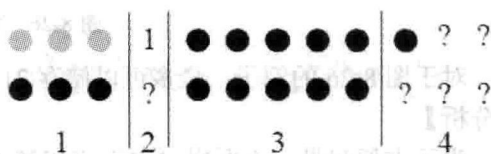


图 8-25 赛程安排的4个阶段

阶段1：首先需要尽量“消灭”黑色队，即依次考虑每一个黑色队，选一个能打败且还没安排对手（称为“配对”）的灰色队。这个阶段结束后，灰色队和黑色队都可能有一些没配对，但有一点是肯定的：已配对的灰色队足以打败现在的所有黑色队。也就是说，对于任意黑色队（不管有没有配对），都至少会输给一支已配对的灰色队。

阶段2：接下来给1号队任选一个能打败的。这个选择一定可以成功，否则说明1号队能打败的队伍不到一半，和假设矛盾。

阶段3：把剩下的黑色队伍任意配对，任它们“自相残杀”，不管谁赢都无所谓。注意，如果前两个阶段结束后没有配对的黑色队伍有奇数个，阶段3之后会有一支黑色队留到第4阶段。

阶段4：剩下的队伍（可能需要加上阶段3后剩下的一支黑色队）任意配对。

下面看这一轮结束后，题目中的各个条件是否依然满足。

条件1：粗略地说，阶段1中的黑色队全军覆没，且阶段3中会消灭一半黑色队，所以总共至少消灭了一半的黑色队。一轮比赛之后，队伍总数减半，而黑色队数目也减半，因此条件1仍满足。细心的读者可能会说：如果阶段4中有一支黑色队，而阶段1完全不存在，则消灭的黑色队不到一半。幸运的是，这样的情况并不存在，因为根据条件2，灰色队伍至少有一支（但有可能只有一支——即这只强大的灰色队可以消灭所有黑色队）。

条件2：此条件之前已经证明过了，阶段1中灰色队伍联合起来可以打败所有黑色队伍，而这些灰色队伍全都晋级到下一轮。

这样就成功解决了本题。

例题 8-18 洞穴 (Cave, ACM/ICPC CERC 2009, UVa1442)

一个洞穴的宽度为 n ($n \leq 10^6$) 个片段组成。已知位置 $[i, i+1]$ 处的地面高度 p_i 和顶的高

度 s_i ($0 \leq p_i < s_i \leq 1000$)，要求在这个洞穴里储存尽量多的燃料，使得在任何位置燃料都不会碰到顶（但是可以无限接近），如图 8-26 所示。

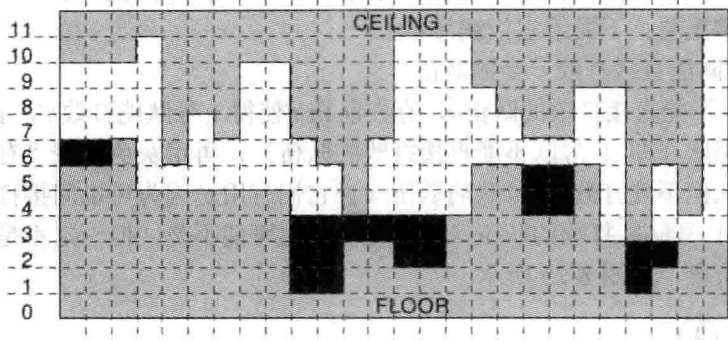


图 8-26 洞穴问题示意图

对于图 8-26 的例子，最多可以储存 21 单位的燃料。

【分析】

为了方便起见，下面用“水”来代替题目中的燃料。根据物理定律，每一段有水的连续区间，水位高度必须相等，且水位必须小于等于区间内的最低天花板高度，因此位置 $[i, i+1]$ 处的水位满足 $h \leq s_i$ ，且从 (i, h) 出发往左右延伸出的两条射线均不会碰到天花板（即两条射线将一直延伸到洞穴之外或先碰到地板之间的“墙壁”）的最大 h 。如果这样的 h 不存在，则规定 $h = p_i$ （也就是“没水”）。

这样，可以先求出“往左延伸不会碰到天花板”的最大值 $h_1(i)$ ，再求“往右延伸不会碰到天花板”的最大值 $h_2(i)$ ，则 $h_i = \min\{h_1(i), h_2(i)\}$ 。根据对称性，只考虑 $h_1(i)$ 的计算。

从左到右扫描。初始时设水位 $\text{level} = s_0$ ，然后依次判断各个位置 $[i, i+1]$ 处的高度。

- 如果 $p[i] > \text{level}$ ，说明水被“隔断”了，需要把 level 提升到 p_i 。
- 如果 $s[i] < \text{level}$ ，说明水位太高，碰到了天花板，需要把 level 下降到 s_i 。
- 位置 $[i, i+1]$ 处的水位就是扫描到位置 i 时的 level 。

不难发现，两次扫描的时间复杂度均为 $O(n)$ ，总时间复杂度为 $O(n)$ 。

例题 8-19 贩卖土地 (Selling Land, ACM/ICPC NWERC 2010, UVa 12265)

输入一个 $n \times m$ ($1 \leq n, m \leq 1000$) 矩阵，每个格子可能是空地，也可能是沼泽。对于每个空地格子，求出以它为右下角的空矩形的最大周长，然后统计每个周长出现了多少次。图 8-27 中标注了 3 个位置的最大空矩形，其周长分别是 6, 10, 12。如果统计完所有 20 个空地，答案是 6×4 （表示周长为 4 的矩形有 6 个）、 5×6 、 5×8 、 3×10 、 1×12 。

【分析】

按照从上到下的顺序处理每一行，在每一行中从左到右处理每个格子（以下称为“当前格”），找出以该格子为右下角的最大周长矩形（以下简称最优矩形）。只要找到了以每个格子为右下角的最优矩形，本题就可以得到解决。

如图 8-28 所示，当前行是图的最下行，当前列是图的最右列（后同）。假定“当前格”已经固定，则只需要再确定一个左上角，就可以得到一个矩形。例如，把格子 A 作为左上

角, 会得到一个矩形 (以下简称矩形 A), 用粗线标出。黑色长条表示题目中的沼泽, 它们上面的格子不影响答案, 因此没有画出。阴影格子表示该区域无法和当前格构成矩形 (更无法构成最优矩形), 因此可以等同于沼泽处理。换句话说, 可以用数组 $height$ 来描述图 8-28 中的图形, 其中 $height[i]$ 表示第 i 列的空地高度。每次“当前行”往下移时, 可以用 $O(m)$ 时间更新 $height$ 数组。

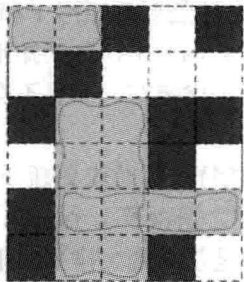


图 8-27 3 个位置的最大空矩形

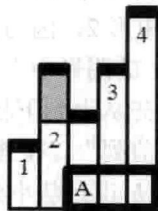
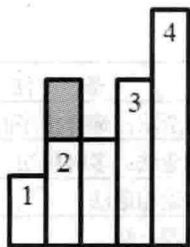


图 8-28 当前行与当前列

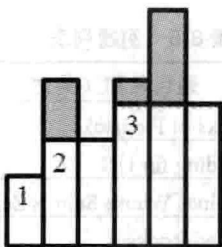
下面考虑图 8-28 中的最优矩形。最优矩形有可能是矩形 A 吗? 不可能, 因为矩形 2 肯定比矩形 A 优 (想一想, 为什么)。矩形 1、2、3、4 哪个最大呢? 在不标明尺寸的情况下无法知道, 需要算一算。不难发现, 在不标明尺寸的情况下, 最优矩形只可能是矩形 1、2、3、4 四者之一。

现在假定“当前行”固定, 而“当前列”往右移动 (最左列编号为 1)。如图 8-29 所示, 最优矩形左上角可能的位置会发生变化。

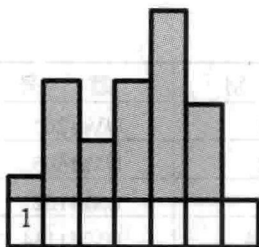
在图 8-29 (a) 中, 最优矩形有 4 种可能, 用 1~4 标记。当前列往右移动一列时, 矩形 4 消失了, 而矩形 3 的高度也变小了 (如图 8-29 (b) 所示)。而当前列再移动一格时, 矩形 2 和矩形 3 都消失了, 矩形 1 也变矮了 (如图 8-29 (c) 所示)。



(a)



(b)



(c)

图 8-29 移动当前列

这就提示要保留最优矩形左上角可能出现的所有位置, 每个位置记为 (c, h) , 表示最左列为 c , 高度为 h 。不难发现, 当 c 从小到大排列时, h 也是从小到大排列的。是不是似曾相识?

没错, 这个“双重有序”的结构和例题“防线”是完全一样的, 不过其他部分有些差别。在例题“防线”中, 需要用二分查找来找到想要的元素, 不过在本题中, 不是要找一个元素, 而是要找所有元素的最大值。这里的“最大”是指以 (c, h) 为左上角、当前格子为右下角的矩形周长最大。格子 (c, h) 所对应的矩形周长为 $2(c_0 - c + 1 + h)$, 其中 c_0 是当前列。不难